SCHOOL OF NATURAL APPLIED SCIENCE

Department of Computer Science and Information Technology (CSIT)

PROJECT – NPRJ740

Advanced ICT Diploma – 2026

CivicLink

Municipal Reporting & Community Intelligence Platform

Phase 2 – System Design & Project Management

Document Name:	CivicLink – Phase 2 System Design Report
System Name:	CivicLink
Student Name:	NTHOROANE KP 202328762
Supervisor:	Dr. Tite Tuyikeze
Submission Date:	01 May 2026

Table of Contents

1. System Requirements	3
1.1 Non-Functional Requirements	3
1.2 Functional Requirements (Product Features)	4
2. System Modelling	5
2.1 Context Diagram	5
2.2 Architectural Design	6
2.3 Use Case Diagram	7
2.4 Activity Diagram	8
2.5 State Diagram	9
2.6 Class Diagram	10
2.7 Sequence Diagram	11
3. Project Management	12
3.1 Work Breakdown Structure	12
3.2 Gantt Chart	14

1. System Requirements

1.1 Non-Functional Requirements

Non-functional requirements define the quality attributes and operational constraints that CivicLink must satisfy. These requirements ensure the system is dependable, secure, and performs appropriately for the Sol Plaatje municipal context.

Attribute	Requirement	Rationale / Measurement
Reliability	The system must maintain 99.5% uptime, with automated failover on critical services.	Sol Plaatje residents depend on real-time incident reporting, especially during emergencies such as strikes or floods.
Availability	CivicLink must be accessible 24 hours a day, 7 days a week. Planned maintenance must not exceed 2 hours per month.	Service delivery incidents (e.g. burst pipes) occur at any time. The system must always be reachable.
Security	All data must be encrypted in transit (TLS 1.3) and at rest. Role-based access control (RBAC) must be enforced across all user types.	Reports may contain sensitive location data and photos. Unauthorised access to admin functions must be prevented.
Maintainability	The codebase must follow modular architecture with documented APIs. Bug patches must be deployable within 4 hours of identification.	Firebase Cloud Functions enable hot-redeployment without downtime. Modular React components allow isolated updates.
Portability	The Resident PWA must be installable on Android 8+ and iOS 13+ devices. The Admin Portal must function on Chrome, Firefox, and Edge.	Kimberley residents predominantly use Android devices. The PWA eliminates the need for app store distribution.
Performance	Reports must appear on the admin dashboard within 5 seconds of submission. Map pins must render within 3 seconds on a 3G connection.	Firestore real-time listeners ensure near-instant data propagation. Google Maps SDK optimises tile loading for low-bandwidth environments.
Scalability	The system must support at least 10,000 concurrent users and 1,000 simultaneous report submissions without degradation.	Firebase auto-scales horizontally. During high-stress events (e.g. public protests), report volumes may spike significantly.
Usability	The resident interface must be operable by users with basic smartphone literacy. Key actions must be completable in 3 steps or fewer.	Sol Plaatje's community includes residents with limited digital literacy. A simplified UI reduces barriers to reporting.

1.2 Functional Requirements (Product Features)

Functional requirements describe what CivicLink must do — the specific behaviours and features the system must provide to each actor.

Resident Features

- Submit incident reports with category selection, free-text description, automatic GPS location capture, and optional photo/video upload.
- View a live community map showing all active reports near the user's location, filterable by category, date, and severity.
- Track the status of submitted reports in real time (Submitted → Under Review → In Progress → Resolved).
- Receive geo-targeted push notifications for area-specific alerts broadcast by the municipality.
- View the public alerts feed including service disruptions, road closures, protest warnings, and emergency announcements.

Municipality Official Features

- View all incoming reports on a centralised admin dashboard with filtering, sorting, and search capabilities.
- Assign reports to specific municipal departments (Water & Sanitation, Electricity, Roads, Waste Management, Public Safety).
- Update report status and add internal notes or resolution comments.
- Create and broadcast geo-targeted alerts to residents in defined geographic areas via Firebase Cloud Messaging.
- Access an analytics dashboard displaying issue frequency heatmaps, average resolution times, department performance, and open incident counts.

Field Worker Features

- Receive digitally assigned tasks with full report details including GPS location, description, and photo evidence.
- Update task status (Accepted → In Progress → Completed) directly from the mobile interface.
- Add completion notes and photographic proof of resolution.

System Admin Features

- Create, deactivate, and manage user accounts across all roles (Resident, Official, Field Worker, Admin).
- Configure system settings including notification templates, category definitions, and department assignments.
- Monitor system health, active sessions, and error logs via the Firebase console.

2. System Modelling

2.1 Context Diagram

The Context Diagram below provides the highest-level view of CivicLink, showing the system as a single process and all external entities that interact with it. Data flows indicate what information passes between the system and each external actor or service.

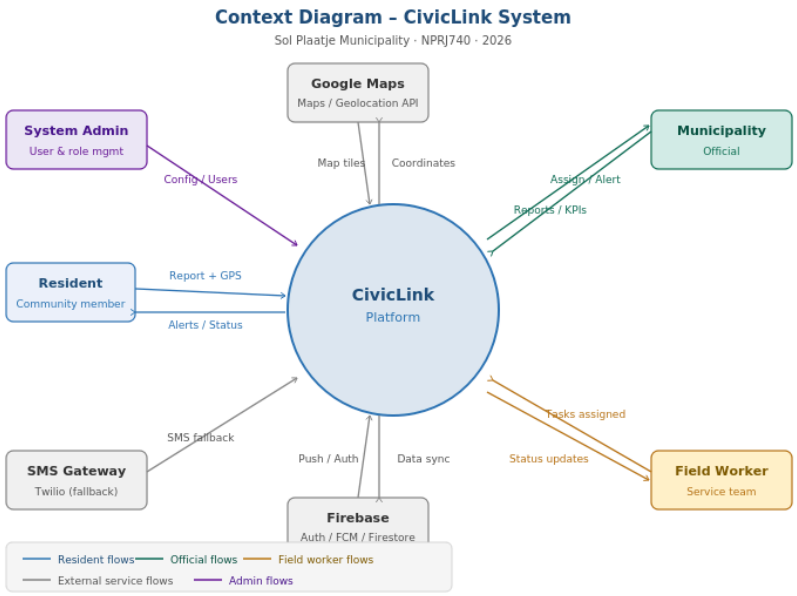


Figure 1: CivicLink Context Diagram – Sol Plaatje Municipality

The five external entities are: (1) Resident, who submits reports and receives alerts and status updates; (2) Municipality Official, who sends assignments and receives reports and analytics; (3) Field Worker, who receives tasks and sends status updates back; (4) Firebase (Auth, FCM, Firestore), which provides authentication, data persistence and push notifications; and (5) Google Maps API, which supplies map tiles and processes GPS coordinates.

2.2 Architectural Design

CivicLink follows a three-layer client-server architecture. This pattern separates presentation, business logic, and data concerns, enabling independent development, testing, and scaling of each layer.

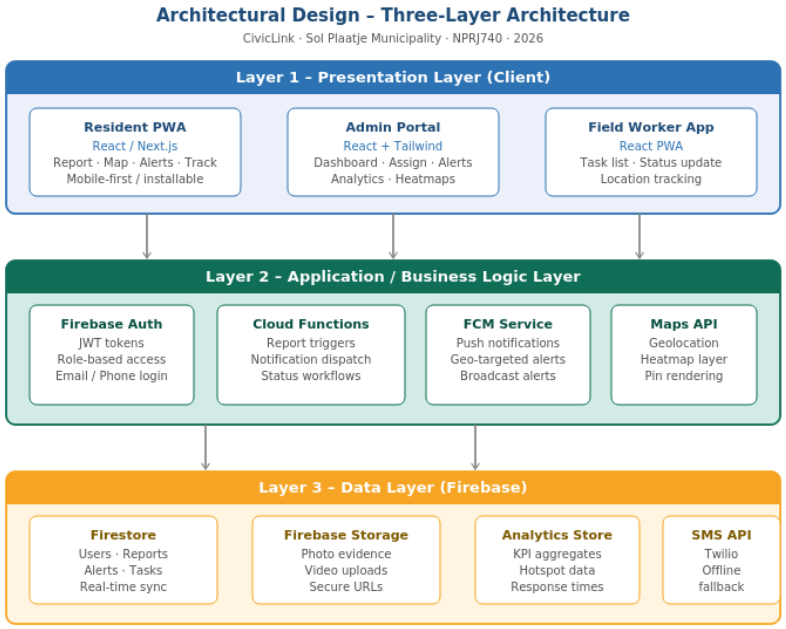


Figure 2: CivicLink Architectural Design – Three-Layer Model

Layer 1 – Presentation Layer

The presentation layer consists of three client-facing interfaces built with React and Next.js: the Resident Progressive Web App (mobile-first, installable on Android/iOS), the Municipality Admin Portal (desktop dashboard with Tailwind CSS), and the Field Worker App (task-focused mobile interface). All three interfaces communicate with the application layer via Firebase SDKs.

Layer 2 – Application / Business Logic Layer

This layer is implemented using Firebase Cloud Functions (Node.js serverless functions) that handle all business logic: report validation and deduplication, notification dispatch via FCM, status workflow management, and geo-targeting of alerts. Firebase Authentication enforces role-based access control across all clients.

Layer 3 – Data Layer

The data layer comprises Firestore (NoSQL real-time database storing Users, Reports, Alerts, Assignments, and Comments collections), Firebase Storage (binary media — photos and videos attached to reports), and the Analytics data store (Firestore aggregations used to power heatmaps and KPI dashboards). A Twilio SMS gateway provides offline fallback for residents without smartphone access.

2.3 Use Case Diagram

The Use Case Diagram captures the interactions between CivicLink's four primary actors — Resident, Municipality Official, Field Worker, and System Admin — and the system's core functional use cases.

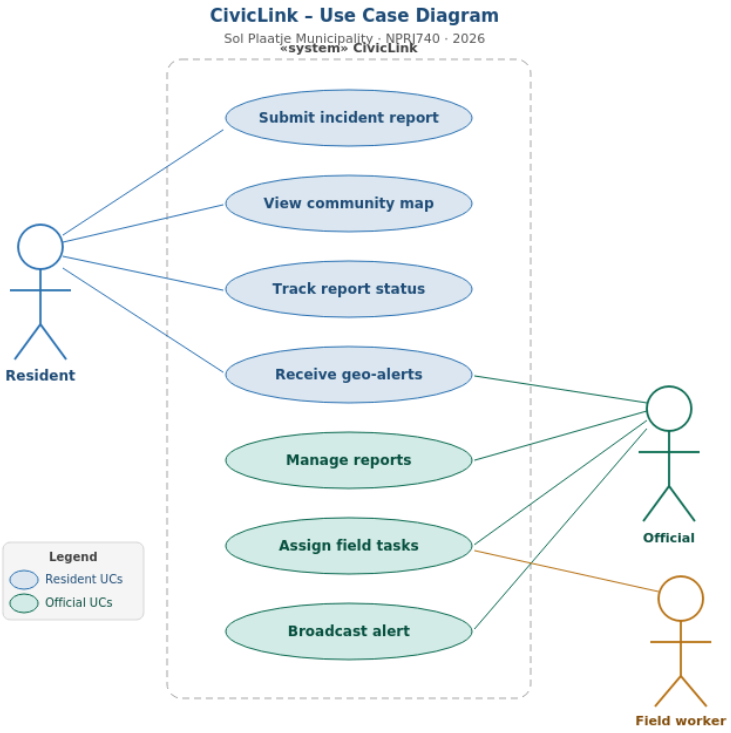


Figure 3: CivicLink UML Use Case Diagram

The Resident interacts with four use cases: Submit Incident Report, View Community Map, Track Report Status, and Receive Geo-Alerts. The Municipality Official interacts with Manage Reports, Assign Field Tasks, Broadcast Alert, Receive Geo-Alerts, and View Analytics. The Field Worker interacts exclusively with Assign Field Tasks — receiving and updating task status.

2.4 Activity Diagram

The Activity Diagram below models the workflow of the primary CivicLink process: incident report submission. It uses swimlanes to delineate actions across three participants — the Resident, the CivicLink system, and the Municipality Official — and includes a decision node for duplicate detection.

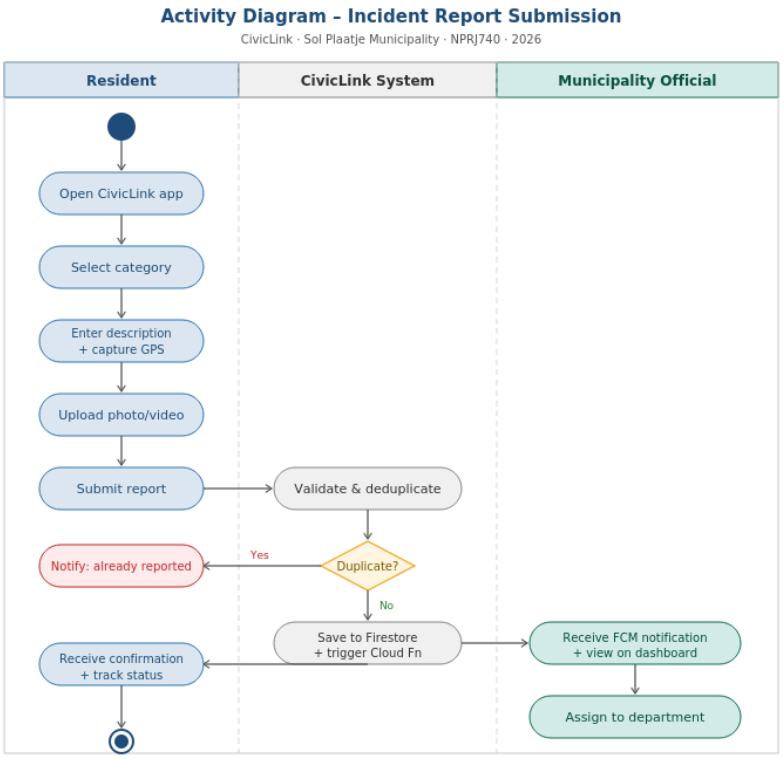


Figure 4: Activity Diagram – Incident Report Submission Workflow

The flow begins when the Resident opens the app and selects a category (e.g. burst pipe, illegal dumping, road blockage). After entering a description and allowing GPS capture, the resident uploads optional photo evidence and submits the report. The system validates the submission and checks for duplicates — if a duplicate is detected, the resident is notified and the flow terminates. If unique, the report is saved to Firestore and a Cloud Function triggers an FCM push to the Municipality Official. The resident receives a confirmation and can track the report's status. The official reviews and assigns the report to the appropriate department.

2.5 State Diagram

The State Diagram models all possible states of a CivicLink incident report throughout its lifecycle, along with the events and actions that trigger state transitions.

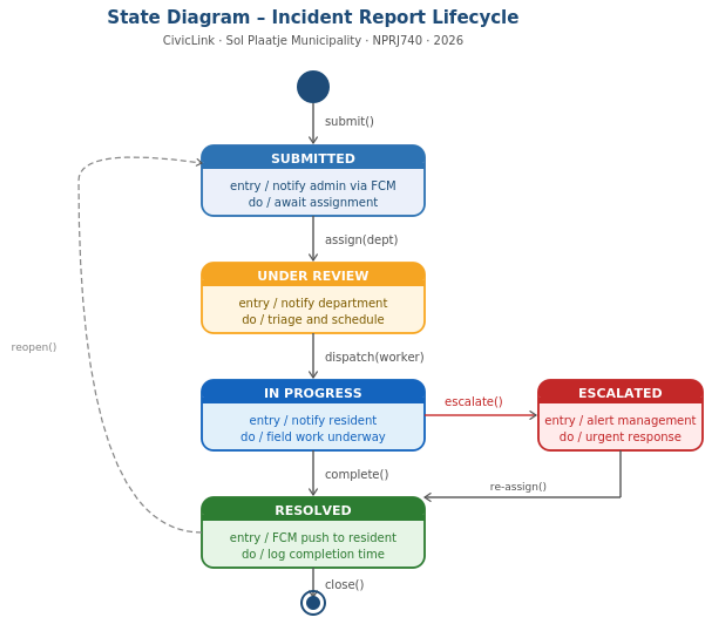


Figure 5: State Diagram – Incident Report Lifecycle

An incident report begins in the SUBMITTED state upon creation, where an FCM notification is dispatched to the admin. Following triage, it transitions to UNDER REVIEW, then to IN PROGRESS once a field worker is dispatched. If the issue exceeds normal resolution parameters, it transitions to ESCALATED, which triggers a management alert before the report is re-assigned and returns to IN PROGRESS. Upon successful resolution by the field worker, the report moves to RESOLVED, triggering a push notification to the resident. A re-open event — such as a recurring burst pipe — returns the report to SUBMITTED. The report is formally closed via a close() event from the resolved state.

2.6 Class Diagram

The Class Diagram defines the object-oriented structure of CivicLink's data model, showing classes, their attributes and methods, and the relationships between them. This directly informs the Firestore collection structure.

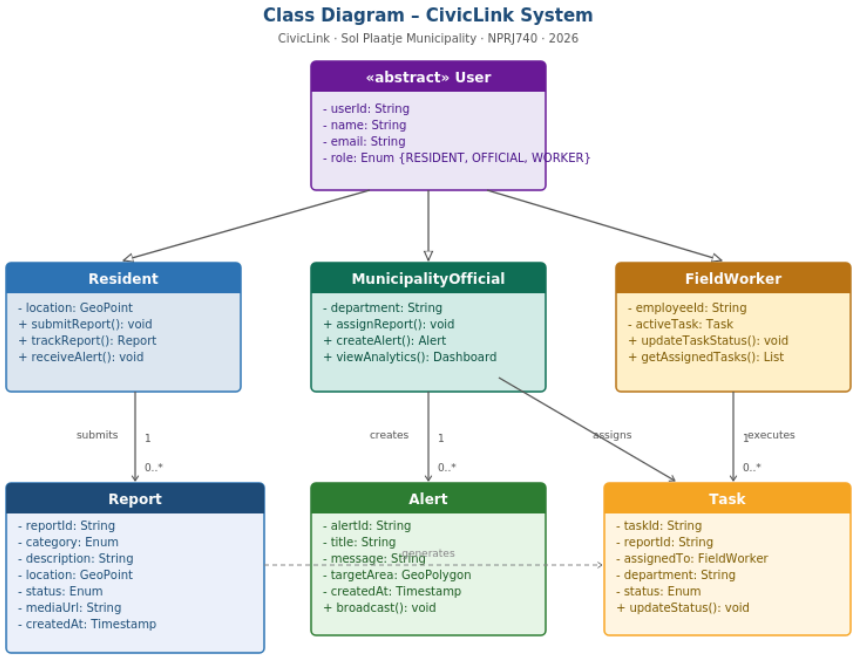


Figure 6: Class Diagram – CivicLink System

The abstract User class is the parent of three specialised subclasses: Resident, MunicipalityOfficial, and FieldWorker — each inheriting base identity attributes and extending them with role-specific methods. The Report class is the central entity, associated with Resident (1 Resident submits 0..* Reports), with Task (1 Report generates 1 Task), and with Alert (indirectly via the broadcast mechanism). The MunicipalityOfficial creates Alerts and assigns Tasks. The FieldWorker executes 0..* Tasks. Composition relationships ensure that Tasks and Reports share the same lifecycle within the platform.

2.7 Sequence Diagram

The Sequence Diagram illustrates the time-ordered interactions between CivicLink's key objects during the complete report submission and resolution lifecycle, including the asynchronous Firebase messaging flows.

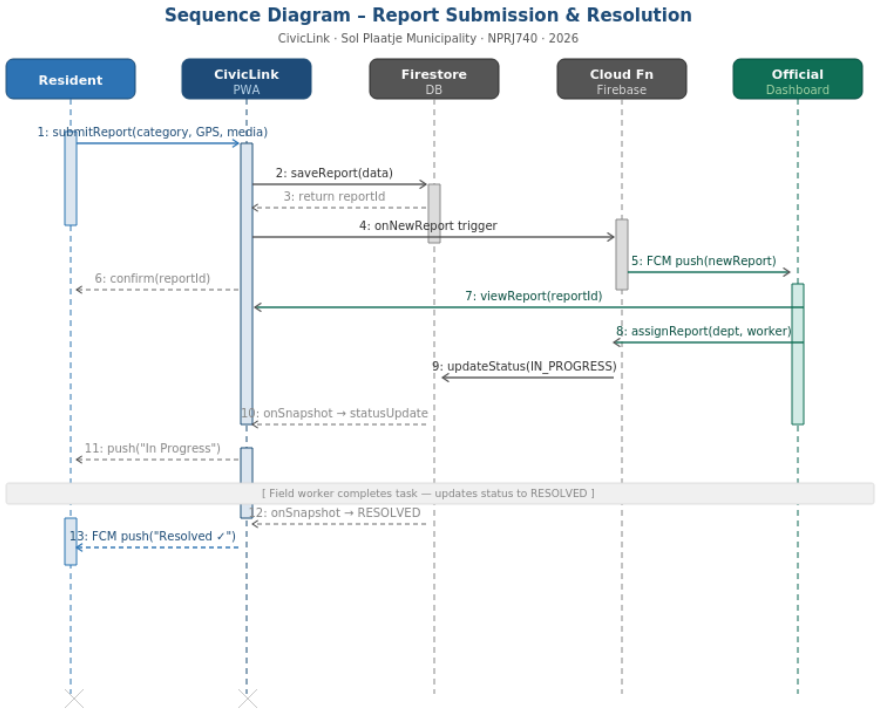


Figure 7: Sequence Diagram – Report Submission & Resolution

The sequence begins with the Resident calling submitReport() on the PWA, which persists the data to Firestore and receives a reportId confirmation. A Firestore Cloud Function trigger fires and dispatches an FCM push to the Municipality Official. The Official reviews the report via the dashboard and calls assignReport(), which updates the Firestore status to IN_PROGRESS via another Cloud Function. A Firestore real-time listener propagates this change to the PWA, which notifies the resident. Once the Field Worker completes the task and updates the status to RESOLVED, the same listener-push mechanism delivers the final FCM notification to the resident confirming resolution.

3. Project Management

3.1 Work Breakdown Structure (WBS)

The Work Breakdown Structure below decomposes the CivicLink project into four phases with clearly defined tasks. Each task includes its deliverable(s), milestone, resources, dependencies, constraints, and associated risks with contingencies.

ID	Task	Deliverable(s)	Milestone	Resources	Dependencies	Risks & Contingencies
Ph.1	**PHASE 1: PLANNING & ANALYSIS**					
1.1	Requirements Gathering	Requirements specification document; interview transcripts	Requirements doc approved (W2)	Student developer; Dr. Tuyikeze (supervisor)	None	Risk: Stakeholder unavailability. Contingency: Use secondary sources and publicly available municipality reports.
1.2	Stakeholder Interview	Interview questionnaire; completed Q&A transcript	Interview conducted (W3)	Student developer; municipality official (mock)	1.1 complete	Risk: Inaccurate responses. Contingency: Cross-validate with online municipal service delivery reports.
1.3	System Analysis & Phase 1 Report	Phase 1 report; problem statement; ICT gap analysis	Phase 1 submitted (W4)	Student developer; supervisor feedback	1.1, 1.2 complete	Risk: Scope creep. Contingency: Strictly limit Phase 1 to analysis; defer design to Phase 2.
Ph.2	**PHASE 2: SYSTEM DESIGN**					
2.1	UI/UX Wireframes	Figma wireframes for Resident PWA and Admin Portal	Wireframes approved (W5)	Student developer; Figma (free tier)	1.3 complete	Risk: UI changes post-approval. Contingency: Freeze wireframes at milestone; document

						change requests.
2.2	Database Schema Design	Firestore collection schema; index definitions; data flow diagram	Schema document approved (W6)	Student developer; Firebase documentation	2.1 in progress	Risk: Schema refactoring during development. Contingency: Design with extensibility in mind; use subcollections sparingly.
2.3	System Diagrams	Context, Architecture, Use Case, Activity, State, Class, Sequence diagrams	All diagrams reviewed (W7)	Student developer; draw.io / SVG tooling	2.1, 2.2 in progress	Risk: Diagram inconsistency with implementation. Contingency: Review diagrams against code during development sprints.
2.4	Phase 2 Report	Phase 2 design report with all diagrams and WBS/Gantt	Phase 2 submitted (W8)	Student developer; supervisor	2.1–2.3 complete	Risk: Formatting errors. Contingency: Validate document against rubric before submission.
Ph.3	**PHASE 3: DEVELOPMENT**					
3.1	Firebase Project Setup	Firebase project; Auth configured; Firestore rules; Storage bucket	Firebase environment live (W8)	Student developer; Firebase free tier	2.2 complete	Risk: Firebase quota limits during development. Contingency: Use Firestore emulator for local testing.
3.2	Resident PWA	Fully functional PWA: report submission, map view, alerts, status tracking	PWA feature-complete (W11)	Student developer; React; Next.js; Google Maps API	3.1 complete	Risk: Google Maps API billing. Contingency: Cap API calls; use static maps for non-critical views.
3.3	Admin Dashboard	Municipality admin portal: incident table, assignment, alert	Admin portal feature-complete (W12)	Student developer; React; Tailwind	3.1 complete	Risk: Complex state management. Contingency: Use Redux or

		creation, analytics		CSS; Recharts		Zustand for predictable state.
3.4	Cloud Functions & FCM	Report triggers; notification dispatch; status workflows; geo-alert broadcasting	Cloud Functions deployed (W13)	Student developer; Firebase Cloud Functions; FCM	3.1 complete	Risk: Cold start latency. Contingency: Minimise function size; use minimum instances setting.
Ph.4	**PHASE 4: TESTING & DEPLOYMENT**					
4.1	User Acceptance Testing	UAT test plan; test case results; bug report log	UAT complete (W15)	Student developer; 3–5 test users; Firebase test environment	3.2–3.4 complete	Risk: Critical bugs discovered late. Contingency: Prioritise P1 bugs; defer P3 cosmetic issues to post-launch.
4.2	Bug Fixing & Refinement	Updated codebase; resolved issue log; regression test results	Stable build approved (W15)	Student developer	4.1 complete	Risk: Insufficient time for fixes. Contingency: Scope fixes to showstopper issues only within timeline.
4.3	Deployment & Final Report	Deployed CivicLink instance; final project report	System live; report submitted (W16)	Student developer; Firebase Hosting; supervisor	4.2 complete	Risk: Deployment configuration errors. Contingency: Use Firebase Hosting CI/CD pipeline with preview channels.

3.2 Gantt Chart

The Gantt chart below illustrates the CivicLink project schedule across 16 weeks (May – August 2026). Tasks are organised into four phases, with diamond markers indicating key project milestones. The chart highlights dependencies between phases and ensures development activities overlap appropriately to meet the submission timeline.

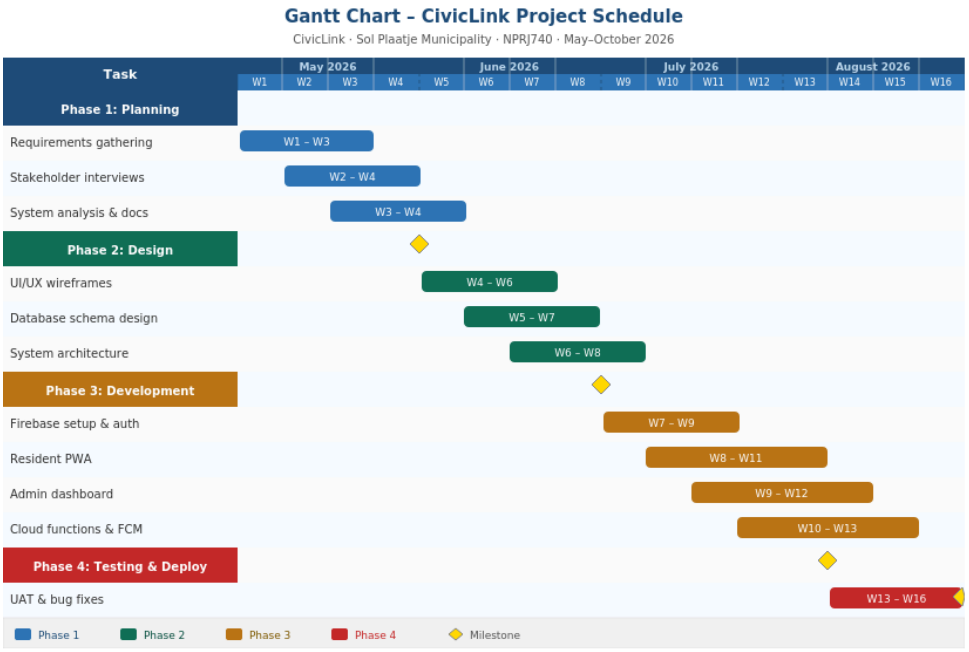


Figure 8: Gantt Chart – CivicLink Project Schedule (May – August 2026)

Phase 1 (Planning & Analysis) spans Weeks 1–4 and establishes the project foundation through requirements gathering, stakeholder interviews, and system analysis. Phase 2 (System Design) runs Weeks 4–8 and overlaps with the tail of Phase 1, ensuring design begins as requirements are finalised. Phase 3 (Development) is the longest phase, spanning Weeks 7–13, with the four development workstreams (Firebase setup, Resident PWA, Admin Dashboard, Cloud Functions) running in staggered parallel. Phase 4 (Testing & Deployment) occupies Weeks 13–16, with UAT and bug fixing preceding the final deployment and report submission.

The milestone at Week 4 marks Phase 1 report submission (1 April 2026). The milestone at Week 8 marks Phase 2 report submission (1 May 2026). The milestone at Week 13 marks development freeze, and the final milestone at Week 16 marks system go-live and final project submission.

— End of Phase 2 Report —